

The Spark UI: skew, shuffle and spill

Finding the longest stage, reading Summary Metrics, and telling the three bottlenecks apart.

Look at the **distribution**, not the total. A stage taking twenty minutes tells you nothing. Twenty minutes where one task took eighteen tells you everything.

Where to look

01

Stages tab

Sort by duration,
descending.

02

The longest stage

Open it.
Everything else is
noise until you
understand this
one.

03

Check for spill

Spill stats
appear at the
top of the stage
page.

04

Summary Metrics

Scroll down.
Compare Max
duration to the
75th percentile.

05

Conclude

Skew, spill, or
neither — and
neither points
elsewhere.

FOLLOW THE PUBLISHED ORDER

This is Databricks' own sequence: spill first, then skew, and if neither, go back to the job page.

The stage columns

COLUMN	WHAT IT MEASURES
Duration	How long the stage took
Input	Bytes read from storage
Output	Bytes written to storage
Shuffle read	Bytes and records read, locally and from remote executors
Shuffle write	Bytes and records written to disk for a future shuffle

ON THE EXAM

Shuffle read and write are how you see a shuffle at all. A stage boundary exists because of one.

Spill happens when Spark runs low on **execution memory** — the memory used for shuffles, joins, sorts and aggregations — and starts moving data to disk.

Two spill numbers

- ◆ **Shuffle spill (memory)** is the size of the deserialised data in memory.
- ◆ **Shuffle spill (disk)** is the size of the serialised form written to disk.
- ◆ **The memory figure is usually much larger**, because deserialised data is bigger than serialised data.
- ◆ **Do not read the gap as an error.** They measure the same spill in two representations.

ON THE EXAM

Any non-zero spill is worth attention. A large memory figure beside a small disk figure is normal, not alarming.

Fixing spill

- ◆ **More partitions first.** Smaller partitions fit in the memory available to process them.
- ◆ **Then more memory,** by choosing a memory-optimised node type.
- ◆ **Reduce the data being shuffled:** filter earlier, select fewer columns.
- ◆ **Spill is a symptom of partitions that are too large** at least as often as of a cluster that is too small.

ON THE EXAM

This is module 15's ordering again. Raising `spark.sql.shuffle.partitions` is usually cheaper than a bigger node type.

In Summary Metrics, compare **Max duration** to the **75th percentile**. If Max is 50% higher, you may be suffering from skew.

What skew actually is

- ◆ **One or a few tasks take much longer than the rest**, because their partitions hold much more data.
- ◆ **The cluster is idle while they finish.** A stage is only done when its slowest task is done.
- ◆ **Usually caused by an uneven key:** one customer with a million orders, or a null that everything joins to.
- ◆ **Look at shuffle read per task too.** Large differences there point at skew as clearly as duration does.

ON THE EXAM

Skew wastes the cluster you are paying for. Ninety-nine executors idle while one finishes is the definition of poor utilisation.

Telling the three apart

WHAT YOU SEE	WHAT IT MEANS
A few tasks much slower than the rest	Data skew
Most tasks slow, fairly evenly	Cluster resources or under-parallelism, not skew
Non-zero spill, high GC time, or OOM	Memory pressure
Large shuffle read and write, many stages	Shuffle-bound work
Large differences in shuffle read per task	Skew again, seen from the data side

ON THE EXAM

The first two rows are the discriminator that matters most. A few slow tasks and all tasks slow have opposite fixes.

Fixing skew

- ◆ **Enable AQE first.** It detects and splits skewed partitions at runtime, and it is on by default from DBR 7.3.
- ◆ **Filter out the pathological key** if it is a null or a placeholder that should not be joined on at all.
- ◆ **Salting:** add a random suffix to the hot key so it spreads across partitions, then aggregate twice.
- ◆ **Repartition on a better key**, or broadcast the small side so no shuffle happens.

ON THE EXAM

Enable AQE before any manual tuning. It resolves a large share of shuffle-partition and join-strategy problems on its own.

Shuffle

- ◆ **Every wide transformation causes one:** join, groupBy, distinct, orderBy.
- ◆ **It moves data across the network** between executors, which is the slowest thing Spark does.
- ◆ **Shuffle read and write on the stage page** tell you how much moved.
- ◆ **Shuffle is not a fault.** The question is whether this one was necessary and whether it moved more than it needed to.

ON THE EXAM

A stage boundary is a shuffle. Counting stages in the UI counts the shuffles you caused.

Reducing shuffle

- ◆ **Broadcast the small side of a join**, which removes the shuffle on the large side entirely.
- ◆ **Filter and select before the wide transformation**, so less data crosses the network.
- ◆ **Avoid unnecessary ordering**. `orderBy` is a full shuffle and is often decoration on an intermediate result.
- ◆ **Right-size shuffle partitions**, remembering AQE coalesces downwards only.

ON THE EXAM

Module 11 and module 15 both apply here. Broadcasting and partition sizing are the same levers, seen from the diagnosis side.

When it is none of the three

- ◆ **No spill and no skew is a real finding**, not a failed investigation.
- ◆ **It points at high input or output** — the stage is simply reading or writing a lot.
- ◆ **Or at under-parallelism**: too few tasks for the cluster, so cores sit idle.
- ◆ **Go back to the job page** and look at the overall shape before drilling in again.

ON THE EXAM

Databricks' guide says this explicitly: if the stage has no spill or skew, look at the stage's I/O next.

Validating a fix

- ◆ **A single faster run is not validation.** Caching and a warm cluster both flatter a repeat.
- ◆ **The underlying metric must move**, not just the wall-clock time.
- ◆ **Check the specific number:** spill to zero, or the Max-to-median ratio down below about two.
- ◆ **Test at full production volume**, because skew often only appears at scale.

ON THE EXAM

Module 15's re-measure discipline, made specific. Confirm the metric you were targeting actually changed.

Tiebreakers from this module

WHEN A QUESTION SAYS	REACH FOR
Max duration far above the 75th percentile	Data skew
Most tasks slow, fairly evenly	Not skew — resources or under-parallelism
Non-zero spill statistics on the stage	Memory pressure. More partitions before more memory.
Spill in memory much larger than on disk	Normal. Deserialised is bigger than serialised.
One customer has millions of rows	A skewed key. AQE, filter, salt, or broadcast.
Large shuffle read and write, many stages	Shuffle-bound. Broadcast, or move less data.
It ran faster once, so the fix worked	Not validation. Check the metric moved.

ON THE EXAM

Seven lines. The first two are the discriminator; the rest follow from it.

A job's longest stage takes 41 minutes. In Summary Metrics, the median task duration is 22 seconds, the 75th percentile is 28 seconds, and the maximum is 37 minutes.

There are 200 tasks. No spill statistics appear on the stage page.

What is the bottleneck?

- ☐ A The cluster is undersized; most tasks are taking too long
- ☐ B Data skew — the maximum is far above the 75th percentile, so one partition holds most of the data
- ☐ C Disk spill, which is why the stage is slow
- ☐ D Too few shuffle partitions for the cluster

◆ PAUSE AND DECIDE

What is the bottleneck?

- A

The cluster is undersized

Most tasks finish in about 22 seconds, which is healthy. If resources were the constraint, the whole distribution would be slow, not one task.
- B

Data skew

Correct. 37 minutes against a 28-second 75th percentile is vastly beyond the 50% threshold. One task is doing almost all the work.
- C

Disk spill

The scenario states no spill statistics appear, and Databricks' guide says that means the stage has no spill.
- D

Too few shuffle partitions

200 partitions with 199 finishing in seconds is not an under-parallelism problem. Adding partitions would not split the one hot key.

THE SIGNAL

41-minute stage, 37-minute maximum task. One task accounts for nearly the whole stage while 199 others idle.

A different stage takes 30 minutes across 200 tasks. The median task duration is 8.4 minutes, the 75th percentile is 9.1 minutes, and the maximum is 9.8 minutes.

The cluster has 16 cores. No spill is reported.

What is the bottleneck?

- ☐ A Data skew, since the maximum is above the median
- ☐ B Not skew — the distribution is even. With 200 tasks on 16 cores, the cluster is under-resourced for the work
- ☐ C Memory pressure, indicated by the long task durations
- ☐ D An excessive number of shuffle partitions creating scheduling overhead

◆ PAUSE AND DECIDE

What is the bottleneck?

A Data skew

9.8 against 9.1 minutes is about 8% above the 75th percentile, far below the 50% threshold. This is one of the most even distributions you will see.

C Memory pressure

Memory pressure shows as spill or GC time, and the scenario states there is no spill. Long tasks alone are not evidence of it.

B Not skew; the cluster is under-resourced

Correct. Every task is slow, evenly. 200 tasks averaging over 8 minutes on 16 cores means roughly 13 waves of work.

D Too many shuffle partitions

Excessive partitions produce many tiny tasks. These tasks take over 8 minutes each, which is the opposite symptom.

THE SIGNAL

This scenario is the mirror of the last one. Same tools, same tab, opposite conclusion — because the distribution is even.

A stage reports shuffle spill of 46 GB in memory and 7 GB on disk. Task durations are fairly even, with the maximum about 20% above the 75th percentile. The stage takes far longer than the team expects.

An engineer proposes moving to a node type with four times the memory.

What should be tried first?

- ☐ A The four-times-memory node type, since spill means insufficient memory
- ☐ B Increase `spark.sql.shuffle.partitions` so each partition is smaller and fits in the memory available
- ☐ C Nothing — 46 GB against 7 GB shows the metrics are inconsistent and cannot be trusted
- ☐ D Enable AQE to split the skewed partitions

◆ PAUSE AND DECIDE

What should be tried first?

A Four times the memory

It would help, at four times the cost, permanently. Spill is as often partitions being too large as memory being too small.

C The metrics are inconsistent

They are consistent. Memory spill measures deserialised data; disk spill measures the serialised form, which is much smaller.

B More shuffle partitions

Correct. Smaller partitions fit in the memory each task has, which removes the spill without changing the cluster.

D Enable AQE for skew

There is no skew — the maximum is 20% above the 75th percentile, under the 50% threshold. AQE would not target this.

THE SIGNAL

46 GB against 7 GB is exactly what you should expect. Deserialised in memory, serialised on disk — the same spill, twice.

An engineer identifies skew in a stage: maximum task duration is 12 times the median. She adds a salting step, redeploys, and re-runs the job immediately. The job completes in 19 minutes rather than 31.

She reports the problem solved. A colleague asks whether she checked Summary Metrics on the re-run.

Is the colleague's question reasonable?

- ☐ A No; a 12-minute improvement is conclusive evidence the fix worked
- ☐ B Yes; an immediate re-run benefits from warm caches, and the Max-to-median ratio must actually have come down
- ☐ C Yes, but only because salting always needs two aggregation passes
- ☐ D No; Summary Metrics do not show skew, only spill

◆ PAUSE AND DECIDE

Is the colleague's question reasonable?

- A

 The improvement is conclusive
Wall-clock time on an immediate re-run is the least reliable measurement available. Warm caches alone can produce a gain this size.
- C

 Yes, but for the salting reason
Salting does involve two passes, and that is not why the colleague's question is reasonable. He is asking about validation, not implementation.
- B

 Yes; check the ratio actually moved
Correct. The metric you targeted must move. If Max-to-median is still 12, the improvement came from something else and will not persist.
- D

 Summary Metrics show only spill
Summary Metrics is precisely where the skew comparison lives. Spill statistics appear at the top of the stage page instead.

THE SIGNAL

If the ratio is still 12 and the job got faster, something else changed. That is worth knowing before you tell everyone it is fixed.

Longest stage. Spill first, then Max against the 75th percentile. **A few slow tasks is skew; all tasks slow is not.**